Classification Assignment – CKD Prediction

1. **Problem Statement:**

   Hospital management wants to predict the Chronic Kidney Disease (CKD) based on several parameters and the provided dataset contains the input and output variables. And the output variable "classification" is categorical (Yes/No). So,

   - Domain → **Machine Learning**
   - Learning → **Supervised Learning**
   - **Classification**

2. **Basic info about the dataset:**

   Total number of rows: **399**

   Total number of columns: **25**

   Independent variables: age, bp, al, su, bgr, bu, sc, sod, pot, hrmo, pcv, wc, rc, sg_b, sg_c, sg_d, sg_e, rbc_normal, pc_normal, pcc_present, ba_present, htn_yes, dm_yes, cad_yes, appet_yes, pe_yes, ane_yes

   Dependent variable: Classification (CKD - Yes or No)

3. **Pre-processing methods:**

   sg, rbc, pc, pcc, htn, dm, cad, appet, pe and ane columns are categorical with nominal data. So, I have converted it into numbers using one-hot encoding.

4. **ML Classification Algorithms comparison:**

   - **Random Forest**

```
[29]: print("RF_Confusion Matrix:\n",cm)

RF_Confusion Matrix:
 [[51  0]
 [ 1 81]]

[30]: print("RF_Classification_Report:\n",clf_report)

RF_Classification_Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133
```

```
[21]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[21]: 0.9997608799617408
```

The Confusion Matrix shows that the Random Forest model correctly classified almost all test cases. Only one observation was misclassified.

TP (True Positive) = 81 → CKD correctly predicted as CKD

TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 1 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 99% accuracy and the ROC_AUC score is 0.9997 which is close to 1, shows that the model gives excellent predictive performance.

- **Decision Tree**:

```
[23]: print("DT_Confusion Matrix:\n",cm)

DT_Confusion Matrix:
 [[50  1]
 [ 4 78]]

[24]: print("DT_Classification_Report:\n",clf_report)

DT_Classification_Report:
              precision    recall  f1-score   support

           0       0.93      0.98      0.95        51
           1       0.99      0.95      0.97        82

    accuracy                           0.96       133
   macro avg       0.96      0.97      0.96       133
weighted avg       0.96      0.96      0.96       133

[19]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[19]: 0.9658058345289334
```

The Confusion Matrix of Decision Tree model shows:

TP (True Positive) = 78 → CKD correctly predicted as CKD

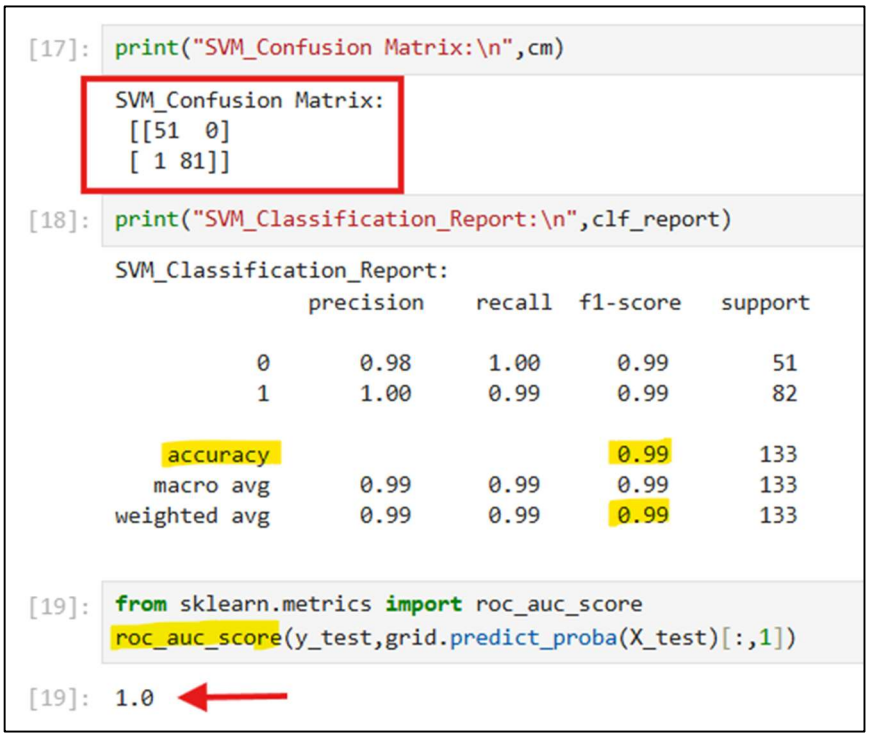
TN (True Negative) = 50 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 1 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 4 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 96% accuracy and the ROC_AUC score is 0.9658 which is near to 1, shows that the model performs well, though slightly lower than the Random Forest model.

- **Support Vector Machine**:



```
[17]: print("SVM_Confusion Matrix:\n",cm)

SVM_Confusion Matrix:
 [[51  0]
 [ 1 81]]

[18]: print("SVM_Classification_Report:\n",clf_report)

SVM_Classification_Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133

[19]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[19]: 1.0
```

The Confusion Matrix shows that the SVM model correctly classified almost all test cases similar to the RF model. Only one observation was misclassified.

TP (True Positive) = 81 → CKD correctly predicted as CKD

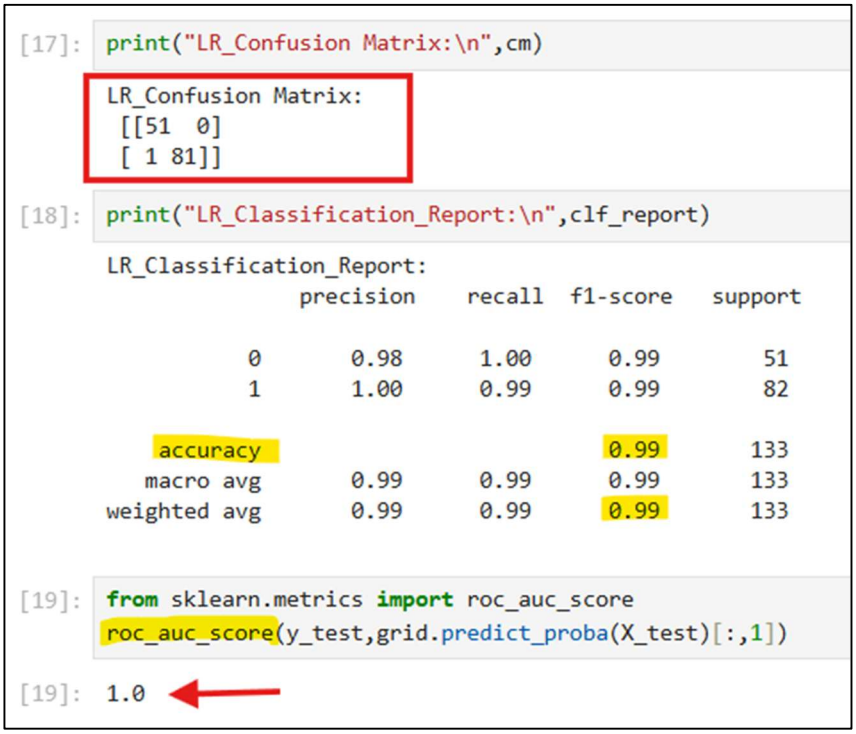
TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 1 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 99% accuracy and the ROC_AUC score is 1.0, shows that the model gives excellent predictive performance and perfect class separation on the test data.

- **Logistic Regression**:



```
[17]: print("LR_Confusion Matrix:\n",cm)

LR_Confusion Matrix:
 [[51  0]
 [ 1 81]]

[18]: print("LR_Classification_Report:\n",clf_report)

LR_Classification_Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        51
           1       1.00      0.99      0.99        82

    accuracy                           0.99       133
   macro avg       0.99      0.99      0.99       133
weighted avg       0.99      0.99      0.99       133

[19]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[19]: 1.0
```

The Confusion Matrix shows that the Logistic Regression model correctly classified almost all test cases similar to the SVM model. Only one observation was misclassified.

TP (True Positive) = 81 → CKD correctly predicted as CKD

TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 1 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 99% accuracy and the ROC_AUC score is 1.0, shows that the model gives excellent predictive performance and perfect class separation on the test data.

- **K Nearest Neighbour**:

```
[17]: print("KNN_Confusion Matrix:\n",cm)

      KNN_Confusion Matrix:
       [[51  0]
       [ 5 77]]

[18]: print("KNN_Classification_Report:\n",clf_report)

      KNN_Classification_Report:
                    precision    recall  f1-score   support

                 0       0.91      1.00      0.95        51
                 1       1.00      0.94      0.97        82

          accuracy                           0.96       133
         macro avg       0.96      0.97      0.96       133
      weighted avg       0.97      0.96      0.96       133

[19]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[19]: 0.998804399808704
```

The Confusion Matrix of KNN model shows:

TP (True Positive) = 77 → CKD correctly predicted as CKD

TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 5 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 96% accuracy and the ROC_AUC score is 0.9988 which is closer to 1, shows that the model performs well.

- **Naïve Bayes**:

  - **Gaussian NB**

```
[17]: print("GaussianNB_Confusion Matrix:\n",cm)

      GaussianNB_Confusion Matrix:
       [[51  0]
       [ 3 79]]

[18]: print("GaussianNB_Classification_Report:\n",clf_report)

      GaussianNB_Classification_Report:
                    precision    recall  f1-score   support

                 0       0.94      1.00      0.97        51
                 1       1.00      0.96      0.98        82

          accuracy                           0.98       133
         macro avg       0.97      0.98      0.98       133
      weighted avg       0.98      0.98      0.98       133


[19]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[19]: 1.0
```

The Confusion Matrix of Gaussian NB model shows:

TP (True Positive) = 79 → CKD correctly predicted as CKD

TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 3 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 98% accuracy and the ROC_AUC score is 1, shows that the model performs well.

- **Multinomial NB**

```
[16]: print("MultinomialNB_Confusion Matrix:\n",cm)

MultinomialNB_Confusion Matrix:
 [[50  1]
 [14 68]]

[17]: print("MultinomialNB_Classification_Report:\n",clf_report)

MultinomialNB_Classification_Report:
              precision    recall  f1-score   support

           0       0.78      0.98      0.87        51
           1       0.99      0.83      0.90        82

    accuracy                           0.89       133
   macro avg       0.88      0.90      0.89       133
weighted avg       0.91      0.89      0.89       133


[18]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[18]: 0.947871831659493
```

The Confusion Matrix of Multinomial NB model shows:

TP (True Positive) = 68 → CKD correctly predicted as CKD

TN (True Negative) = 50 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 1 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 14 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 89% accuracy and the ROC_AUC score is 0.9478. But the false negative prediction is 14 which is higher than other models so this model performs low compared to other models.

- **Bernoulli NB**

```
[16]: print("BernoulliNB_Confusion Matrix:\n",cm)

BernoulliNB_Confusion Matrix:
 [[51  0]
 [ 3 79]]

[17]: print("BernoulliNB_Classification_Report:\n",clf_report)

BernoulliNB_Classification_Report:
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        51
           1       1.00      0.96      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133


[18]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[18]: 0.9966523194643712
```

The Confusion Matrix of Bernoulli NB model shows:

TP (True Positive) = 79 → CKD correctly predicted as CKD

TN (True Negative) = 51 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 0 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 3 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 98% accuracy and the ROC_AUC score is 0.9966, shows that the model performs well.

- **Complement NB**

```
[16]: print("ComplementNB_Confusion Matrix:\n",cm)

ComplementNB_Confusion Matrix:
 [[50  1]
 [14 68]]

[17]: print("ComplementNB_Classification_Report:\n",clf_report)

ComplementNB_Classification_Report:
              precision    recall  f1-score   support

           0       0.78      0.98      0.87        51
           1       0.99      0.83      0.90        82

    accuracy                           0.89       133
   macro avg       0.88      0.90      0.89       133
weighted avg       0.91      0.89      0.89       133


[18]: from sklearn.metrics import roc_auc_score
      roc_auc_score(y_test,grid.predict_proba(X_test)[:,1])

[18]: 0.947871831659493
```

The Confusion Matrix of Complement NB model shows:

TP (True Positive) = 68 → CKD correctly predicted as CKD

TN (True Negative) = 50 → Non-CKD correctly predicted as Non-CKD

FP (False Positive) = 1 → Non-CKD wrongly predicted as CKD

FN (False Negative) = 14 → CKD wrongly predicted as Non-CKD

Classification report says that the model achieved 89% accuracy and the ROC_AUC score is 0.9478. But the false negative prediction is 14 which is higher than other models so this model performs low compared to other models.

5. Summary:

Model	Accuracy	ROC-AUC	False Negative (Missed CKD)
Random Forest	99%	0.9997	1
Decision Tree	96%	0.9658	4
SVM	**99%**	**1**	**1**
Logistic Regression	**99%**	**1**	**1**
KNN	96%	0.9988	5
Gaussian NB	98%	1	3
Multinomial NB	98%	0.9478	14
Bernoulli NB	98%	0.9966	3
Complement NB	98%	0.9478	14

6. Final Model:

- I would choose **SVM or Logistic Regression** model because,
  - Highest accuracy (99%)
  - ROC-AUC = 1.0 (perfect class separation)
  - Only one false negative
- Since this is a medical prediction problem, minimizing false negatives (missing CKD patients) is very important. SVM and Logistic Regression both perform best in this aspect comparing to other models.